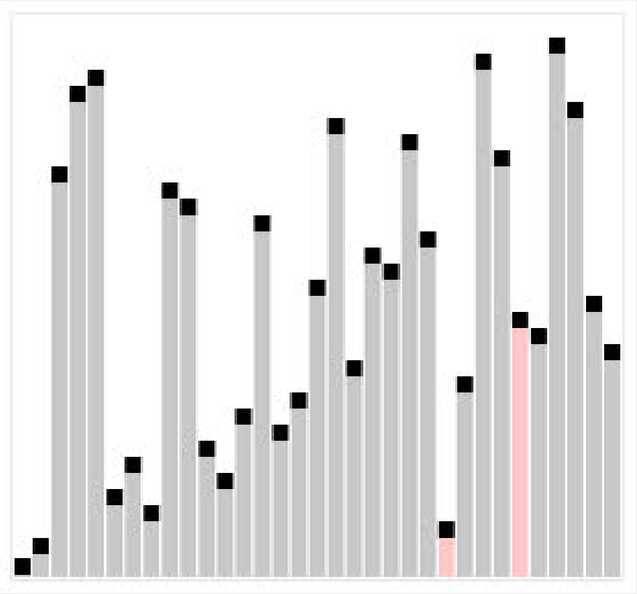
Selection sort

Estudo detalhado do algoritmo, contemplando análise de custo, complexidade de tempo e espaço, e diferentes abordagens de implementação. Os resultados serão ilustrados graficamente e validados por meio de testes realizados durante a apresentação.



Luccas Henrique e Matheus Garcia

Análise Selection Sort

Sumário

Funcionamento

Passo a passo da lógica do algoritmo: a busca iterativa pelo menor elemento e a estratégia de trocas posicionais.

Análise

Interpretação técnica das métricas e performance observada nos testes.

Contexto Geral

Abordaremos os conceitos fundamentais de ordenação. Breve histórico sobre a concepção do algoritmo e sua relevância acadêmica.

Combate

Duelo de performance: benchmarks e gráficos comparativos com o Quick Sort.

Algoritmos de ordenação

Ordenar é o processo de arranjar itens em uma sequência lógica (numérica ou alfabética). Sem isso, tarefas simples seriam impossíveis:

- Busca Eficiente: Algoritmos de busca binária só funcionam em listas ordenadas. Sem ordenação, buscar um nome em um banco de dados de milhões de registros levaria horas em vez de milissegundos.
- Processamento de Dados: Relatórios financeiros, rankings de jogos e sistemas de logística dependem totalmente de dados estruturados.
- Experiência do Usuário: Filtros de "menor preço" em e-commerces ou "postagens mais recentes" em redes sociais são alimentados por esses algoritmos.

Selection Sort

Contexto Geral

1950 e 1960

Não possui um inventor específico ou uma data de criação única, pois é considerado um algoritmo clássico que surgiu e foi formalizado de forma independente por diversos indivíduos.

Objetivo

Seu objetivo principal é fornecer uma solução direta, intuitiva e de implementação simples para a ordenação de dados, atendendo à necessidade prática urgente de organizar informações em sistemas computacionais iniciais, onde a simplicidade conceitual era mais valiosa do que a eficiência máxima.

Redução e Repetição

Selection Sort

Como ele funciona?

Redu_Rep C++

```cpp
for (int i = 0; i < n - 1; i++) { // 'i' define o início da parte desordenada
    // ... o código aqui dentro trabalha apenas de 'i' até o fim
}
```

03

Busca do Menor (varredura)

Selection Sort

Como ele funciona?

Varredura C++

```cpp
for (int j = i + 1; j < n; j++) {
    comparacoes++; // Registro para sua análise
    if (arr[j] < arr[idx_min])
        idx_min = j; // Encontrou um candidato menor
}
```

01

Swap

Swap C++

```cpp
if (idx_min != i) {
    int tmp      = arr[idx_min];
    arr[idx_min] = arr[i];
    arr[i]       = tmp;
    trocas++;
}
```

Selection Sort

Como ele funciona?

02

Selection Sort

Exemplo

Selection Sort

Como foi feito os testes?

Especificações da máquina
- 16 GB DDR4
- I3 11th
- Windows/Linux

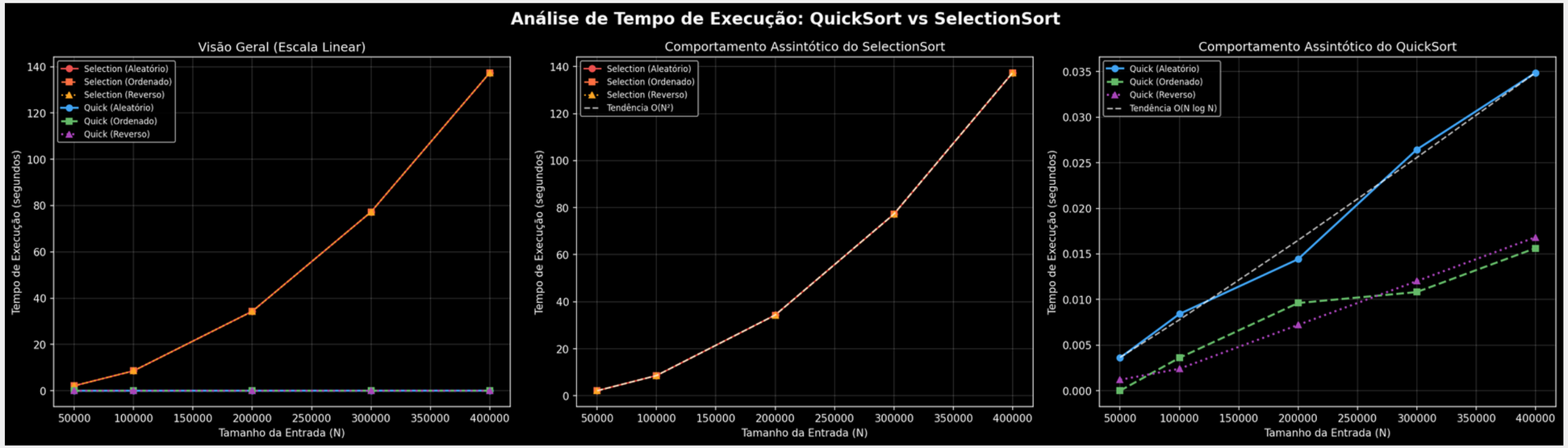
Quantidade de testes
- Algoritmo PCG64.
- Ordenado, inverso e aleatório.
- Script em python para reduzir I/O e consumo de memória.

Bibliotecas
- numpy
- matplotlib
- sys, os, time e subprocess
- logging

Programas ultilizados
- Vscode
- Git

Selection Sort

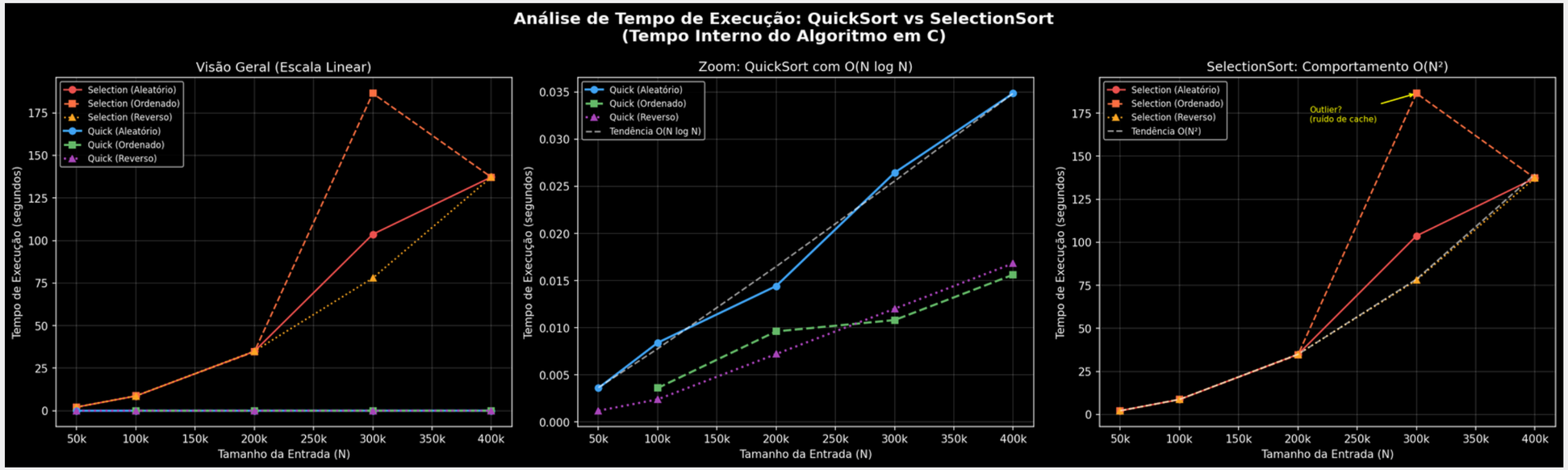
Análise



https://github.com/Mtheusgs/MatheusGarcia_LuccasVieira_ws_AA_RR_2026

Selection Sort

Análise



Selection Sort

Calculo de complexidade

Melhor Caso (O(n^2))

O vetor já está perfeitamente ordenado (ex: [1, 2, 3, 4, 5]).

Caso médio (O(n^2))

Os dados estão distribuídos de forma aleatória.

Pior caso (O(n^2))

O vetor está em ordem inversa (ex: [5, 4, 3, 2, 1]) ou com o menor elemento na última posição.

Selection Sort

Cálculo

Dedução Matemática da Complexidade

O número total de comparações $C(n)$ no algoritmo *Selection Sort* é determinado pela execução de dois laços de repetição aninhados. O objetivo é calcular a quantidade exata de operações fundamentais.

1. Modelagem por Somatórios

O loop externo percorre os índices de $i = 0$ até $n-2$. Para cada iteração de i, o loop interno percorre de $j = i+1$ até $n-1$. Como a comparação possui custo constante, representamos por 1:

$$C(n) = \sum_{i=0}^{n-2} \left(\sum_{j=i+1}^{n-1} 1 \right) \tag{1}$$

2. Resolução do Somatório Interno

Aplicando a propriedade da soma de uma constante $\sum_{k=a}^{b} 1 = (b-a+1)$:

$$\sum_{j=i+1}^{n-1} 1 = (n-1) - (i+1) + 1 = n - i - 1 \tag{2}$$

Substituindo o resultado no somatório externo:

$$C(n) = \sum_{i=0}^{n-2} (n-i-1) \tag{3}$$

3. Expansão e Soma de Gauss

Ao expandir os termos para $i = 0, 1, \ldots, n-2$, obtemos a soma dos primeiros $n-1$ inteiros:

$$C(n) = (n-1) + (n-2) + \cdots + 1 \tag{4}$$

Utilizando a fórmula da Progressão Aritmética $S = \frac{(a_1 + a_n) \cdot k}{2}$:

$$C(n) = \frac{((n-1)+1) \cdot (n-1)}{2} = \frac{n(n-1)}{2} \tag{5}$$

4. Resultado Final e Complexidade

Expandindo a expressão final:

$$C(n) = \frac{1}{2}n^2 - \frac{1}{2}n \tag{6}$$

Na análise assintótica, focamos no termo de maior ordem e desprezamos as constantes. Como o comportamento do algoritmo é idêntico para o melhor e pior caso (sempre percorre todos os elementos):

- **Limite Superior (Big O):** $O(n^2)$
- **Limite Justo (Theta):** $\Theta(n^2)$

5. Prova Formal da Notação Θ

Para provar que $f(n) = \frac{1}{2}n^2 - \frac{1}{2}n$ é $\Theta(n^2)$, devemos encontrar constantes $c_1, c_2 > 0$ e $n_0 \geq 1$ tais que:

$$c_1 \cdot n^2 \leq \frac{1}{2}n^2 - \frac{1}{2}n \leq c_2 \cdot n^2 \tag{7}$$

Limite Superior (O): Para $n \geq 1$, temos que $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$. Portanto, $c_2 = 1/2$ e $n_0 = 1$ satisfazem o limite superior.

Limite Inferior (Ω): Queremos mostrar que $\frac{1}{2}n^2 - \frac{1}{2}n \geq c_1 \cdot n^2$. Fatorando n^2 no lado esquerdo:

$$\frac{1}{2}n^2 - \frac{1}{2}n = n^2 \left(\frac{1}{2} - \frac{1}{2n} \right) \tag{8}$$

Para $n \geq 2$, temos $\frac{1}{2n} \leq \frac{1}{4}$, e portanto:

$$\frac{1}{2} - \frac{1}{2n} \geq \frac{1}{2} - \frac{1}{4} = \frac{1}{4} \tag{9}$$

Logo:

$$n^2 \left(\frac{1}{2} - \frac{1}{2n} \right) \geq \frac{1}{4} \cdot n^2 \tag{10}$$

Escolhendo $c_1 = 1/4$ e $n_0 = 2$, o limite inferior é satisfeito.

Como existem constantes que satisfazem ambos os limites, conclui-se que:

$$f(n) = \Theta(n^2) \tag{11}$$

Quick Sort

Otimizações

01

Dual pivot

Em vez de dividir o array em dois (menores e maiores que o pivô), o código usa dois pivôs.

02

Mediana de 5

O código não pega o primeiro elemento como pivô. Ele olha para 5 pontos diferentes do array (lo, m1, mid, m3, hi).

03

Corte para Insertion Sort (Threshold)

No código, há um #define INSERTION_THRESHOLD 16. Se a partição for menor que 16 elementos, ele para de usar Quick Sort e usa o Insertion Sort.

Quick Sort - Selection Sort

Comparação de resultados

```
10156-[2026-04-27 16:45:55,095]: DEBUG → ============================================================
10156-[2026-04-27 16:45:55,095]: DEBUG → ═══ INICIANDO BATERIA: numeros_400k_reverso.txt ═══
10156-[2026-04-27 16:45:55,181]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 1/13 finalizada.
10156-[2026-04-27 16:45:55,251]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 2/13 finalizada.
10156-[2026-04-27 16:45:55,312]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 3/13 finalizada.
10156-[2026-04-27 16:45:55,385]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 4/13 finalizada.
10156-[2026-04-27 16:45:55,447]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 5/13 finalizada.
10156-[2026-04-27 16:45:55,531]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 6/13 finalizada.
10156-[2026-04-27 16:45:55,618]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 7/13 finalizada.
10156-[2026-04-27 16:45:55,698]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 8/13 finalizada.
10156-[2026-04-27 16:45:55,762]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 9/13 finalizada.
10156-[2026-04-27 16:45:55,839]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 10/13 finalizada.
10156-[2026-04-27 16:45:55,904]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 11/13 finalizada.
10156-[2026-04-27 16:45:55,970]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 12/13 finalizada.
10156-[2026-04-27 16:45:56,033]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 13/13 finalizada.
10156-[2026-04-27 16:45:56,033]: DEBUG → --- RELATORIO DA BATERIA: numeros_400k_reverso.txt ---
10156-[2026-04-27 16:45:56,033]: DEBUG → N de Execucoes: 13
10156-[2026-04-27 16:45:56,033]: DEBUG → Tempo Total (Wall Time): 0.938343s
10156-[2026-04-27 16:45:56,033]: DEBUG → Tempo Medio (Wall Time): 0.072180s
10156-[2026-04-27 16:45:56,033]: DEBUG → Tempo Medio (Interno do C): 0.016827s
10156-[2026-04-27 16:45:56,033]: DEBUG → Soma de Comparacoes (Total): 72538596
10156-[2026-04-27 16:45:56,033]: DEBUG → Media de Comparacoes/Rodada: 5579892.00
10156-[2026-04-27 16:45:56,033]: DEBUG → ============================================================
```

```
5296-[2026-04-27 23:37:10,763]: DEBUG → ============================================================
5296-[2026-04-27 23:37:10,763]: DEBUG → ═══ INICIANDO BATERIA: numeros_400k_reverso.txt ═══
5296-[2026-04-27 23:39:28,250]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 1/13 finalizada.
5296-[2026-04-27 23:41:45,685]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 2/13 finalizada.
5296-[2026-04-27 23:44:03,146]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 3/13 finalizada.
5296-[2026-04-27 23:46:20,574]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 4/13 finalizada.
5296-[2026-04-27 23:48:38,007]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 5/13 finalizada.
5296-[2026-04-27 23:50:57,641]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 6/13 finalizada.
5296-[2026-04-27 23:53:15,074]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 7/13 finalizada.
5296-[2026-04-27 23:55:32,548]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 8/13 finalizada.
5296-[2026-04-27 23:57:50,034]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 9/13 finalizada.
5296-[2026-04-28 00:00:07,543]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 10/13 finalizada.
5296-[2026-04-28 00:02:25,002]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 11/13 finalizada.
5296-[2026-04-28 00:04:42,479]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 12/13 finalizada.
5296-[2026-04-28 00:06:59,898]: DEBUG →   > [numeros_400k_reverso.txt] Execucao 13/13 finalizada.
5296-[2026-04-28 00:06:59,898]: DEBUG → --- RELATORIO DA BATERIA: numeros_400k_reverso.txt ---
5296-[2026-04-28 00:06:59,898]: DEBUG → N de Execucoes: 13
5296-[2026-04-28 00:06:59,898]: DEBUG → Tempo Total (Wall Time): 1789.134902s
5296-[2026-04-28 00:06:59,898]: DEBUG → Tempo Medio (Wall Time): 137.625762s
5296-[2026-04-28 00:06:59,898]: DEBUG → Tempo Medio (Interno do C): 137.335337s
5296-[2026-04-28 00:06:59,898]: DEBUG → Soma de Comparacoes (Total): 1039997400000
5296-[2026-04-28 00:06:59,898]: DEBUG → Medio de Comparacoes/Rodada: 79999800000.00
5296-[2026-04-28 00:06:59,898]: DEBUG → ============================================================
```

Pior caso (vetor ordenado em ordem decrescente)

Análise do Selection Sort

Obrigado

Obrigado pelo seu tempo.

Nós convidamos vocês a acessarem o repositório.